Fixing SDXL VAE Spurious Bright Pixels

Image Editing Techniques and Visual Effects Compositing — Term Project
National Yang Ming Chiao Tung University, Spring 2026

Phecda

Demo: https://www.youtube.com/watch?v=qqUrk4OyEfI
GitHub: https://github.com/megrez33281/ComfyUI-VAE-Noise-Fix

Abstract

Stable Diffusion XL (SDXL) models frequently produce isolated bright pixels and color-shifted spots after VAE decoding, a well-documented side effect of the 4-channel latent space bottleneck combined with aggressive CFG scales. Existing fixes rely on neural-network-based inpainting (e.g., re-running a U-Net pass), which consumes significant VRAM and risks altering already-satisfactory image regions. This project presents ComfyUI-VAE-Noise-Fix, a lightweight ComfyUI custom node that detects and repairs these artifacts entirely through classical computer vision techniques, including Laplacian filtering, median residual analysis, connected-component analysis, CIE-LAB chromaticity verification, and Telea inpainting. The system runs in milliseconds on CPU, requires zero additional model weights, and provides 11 diagnostic preview modes for transparent parameter tuning. A standalone GUI tool and a browser-based mask editor are also included to support interactive workflows.

1. Introduction

The rise of text-to-image diffusion models has brought remarkable creative power to artists and designers, but the pipeline from latent space to pixel space remains an imperfect bridge. In SDXL specifically, the Variational Autoencoder (VAE) that performs final decoding maps a heavily compressed 4-channel representation back to full RGB, and this step frequently introduces what the community calls "fireflies", i.e., isolated pixels or tiny clusters that are drastically brighter, darker, or chromatically shifted relative to their neighbors.

These artifacts are particularly common under certain conditions: high CFG scale values, dark or uniform backgrounds, and high-contrast edges. While they are small in pixel count, they are visually jarring and undermine the perceived quality of generated images. The standard workaround (running an additional neural-network inpainting pass or applying a high-resolution fix) solves the problem but at a steep cost in computation time, VRAM usage, and the risk of inadvertently modifying parts of the image the user is already happy with.

This project takes a different approach. Instead of fighting a pixel-level problem with a billion-parameter model, we observe that these VAE artifacts share a remarkably consistent set of low-level visual signatures (sharp gradient discontinuities, spatial isolation, and chromaticity deviations) that are well within the reach of classical image processing. The result is a five-stage detection pipeline paired with Telea Fast Marching inpainting, packaged as a drop-in ComfyUI custom node with no GPU requirement and sub-second processing time.

2. Problem Analysis

SDXL's VAE operates in a 4-channel latent space with 8× spatial compression. At high CFG scales, the diffusion process often pushes latent values beyond the VAE's smoothly decodable range. When this occurs, the decoder **produces** numerical overflow artifacts that typically manifest in two primary forms:

- **Sharp impulse noise:** single pixels or 2–3 pixel clusters that jump to extreme values (pure white, pure black, or saturated colors) with no gradual transition from the surrounding region. Under a Laplacian operator, these sites produce very high second-derivative energy.
- **Smooth color-block noise:** small patches (3–6 pixels) of uniform but incorrect color, with relatively soft edges. These are harder to catch with gradient-based filters alone because their internal gradients are low; instead, they stand out in terms of color deviation from the local neighborhood.

A robust detector must handle both forms while preserving legitimate small bright features (stars, specular highlights, fine jewelry) that share a superficially similar appearance. This discrimination problem is the central technical challenge of the project.

3. System Architecture

The system is organized into three layers. The core algorithm layer contains 11 single-responsibility modules that implement the detection and repair pipeline. The binding layer adapts the pipeline for ComfyUI (node registration, tensor marshaling) and for a standalone GUI. The frontend layer provides a JavaScript-based interactive mask editor that runs inside the ComfyUI browser interface.

3.1 File Organization

Layer	Responsibility
Algorithm	Energy extraction, thresholding, CCA filtering, LAB verification, dilation, inpainting, overlay rendering, statistics
ComfyUI binding	Node class registration, tensor bridge, parameter widgets
Server API	REST endpoints for the browser mask editor
GUI	Standalone tkinter preview app with zoom, brush editing, statistics HUD
Frontend	In-browser mask painting with undo/redo, zoom, pan

3.2 ComfyUI Node Set

Four nodes are exposed to cover different workflow needs. VAENoiseFix is the all-in-one node that performs detection and repair in a single step. VAENoiseDetector performs detection only and outputs a mask for downstream editing. VAENoiseInpainter takes an image and an externally-provided mask and applies Telea inpainting. VAENoiseMaskEditor sits between the detector and inpainter, staging data for the browser-based mask editor and picking up the user's edits on the next queue.

3.3 Supported Workflows

Workflow A — Fully automatic:

```
[KSampler] → [VAE Decode] → [VAENoiseFix] → [Save Image]
```

Workflow B — Manual mask editing:

```
[VAE Decode] → [VAENoiseDetector] → [VAENoiseMaskEditor] → [VAENoiseInpainter] → [Save Image]
```

4. Detection Pipeline

The detection pipeline consists of five sequential stages. Each stage is implemented as an independent module with a single public method, making it straightforward to test, swap, or extend individual components.

4.1 Stage 1 — Dual-Path Energy Extraction

Two complementary energy maps are computed from the input image. The Laplacian path converts the image to BT.709 grayscale and applies a 3×3 Laplacian kernel (cv2.Laplacian with CV_16S depth to avoid uint8 overflow), then takes the absolute value to produce an energy map that highlights pixels with sharp gradient discontinuities in any direction. This path excels at catching impulse-type noise with hard edges.

The median residual path computes the per-channel absolute difference between the original image and its median-filtered version, then takes the per-pixel channel-wise maximum. The median filter kernel size is derived dynamically from the user's max_noise_size parameter (diameter ≈ ⌈√area⌋, forced odd, minimum 5). Because the median filter is inherently robust to isolated outliers, the residual concentrates energy precisely at locations where a few pixels deviate strongly from their neighborhood, which is the signature of color-block noise that the Laplacian may underestimate.

4.2 Stage 2 — Dual-Threshold Binarization

The two energy maps are binarized and merged via bitwise OR at two different sensitivity levels. At the user-specified sensitivity (e.g., 0.35), the result is the seed mask: only the most energetically extreme pixels survive, marking the cores of potential noise sites. At a lower threshold (25% of user sensitivity, floored at 0.05), the result is the context mask: a broader region that captures the full spatial extent of each candidate blob, including any halo or transitional fringe. The downstream structural filter uses both masks together.

4.3 Stage 3 — Structural Filtering via Connected-Component Analysis

Connected-component analysis (CCA) is applied to the context mask using 8-connectivity. Each resulting blob is tested against three rejection rules:

1. **Isolation rule:** The context-mask area of a blob must not exceed 5 × max_noise_size (resolution-scaled). Blobs larger than this threshold are considered parts of legitimate image structures rather than isolated artifacts.

2. **Seed presence rule:** Each context blob must contain at least one pixel from the seed mask, and the total seed area within the blob must not exceed max_noise_size. This ensures that only blobs with genuinely high-energy cores are retained; low-energy micro-regions that happen to pass the context threshold are discarded.
3. **Compactness rule:** The rotated minimum-area bounding rectangle (cv2.minAreaRect) aspect ratio must be ≤ 3:1. Elongated structures (hair strands, fine lines, scratches) are geometrically incompatible with the roughly isotropic shape expected of VAE noise. To keep this stage fast even on 4K images, an axis-aligned bounding box (AABB) pre-filter quickly classifies obviously compact and obviously elongated blobs, reserving the more expensive rotated-box computation for the ambiguous middle band (typically less than 1% of all blobs).

The output mask is constructed in a single pass using a label lookup table rather than per-blob whole-image comparisons, which is critical for performance when the context mask may contain tens of thousands of connected components.

4.4 Stage 4 — CIE-LAB Chromaticity Verification

This stage addresses the hardest discrimination problem: distinguishing VAE noise from natural bright features like stars, specular highlights, and small reflections. Both categories are small, bright, and isolated, making them structurally identical. The key difference lies in their luminance profile.

The image region around each surviving blob is converted to CIE-LAB color space, which separates luminance (L^*) from chrominance (a^*, b^*) in a perceptually uniform manner. Two checks are performed:

- **Cliff steepness test:** For each blob, we compute internal_drop_ratio = $\frac{L^*_{peak} - \bar{L}^*_{comp}}{L^*_{peak} - \bar{L}^*_{bg}}$. A ratio near 1.0 indicates an impulse: the blob's internal pixels are all close to the peak value with no gradual falloff, like a cliff edge. A ratio near 0.5 indicates a gradual hill, where the blob contains a range of luminance values transitioning smoothly toward the background, characteristic of natural highlights. The acceptance threshold is sensitivity-tuned: 0.6 − 0.2 × sensitivity.
- **Chromaticity distance test:** The Euclidean distance between the blob's mean (a^*, b^*) and the surrounding background's mean (a^*, b^*) is compared against a threshold of 3 + 15 × sensitivity. Blobs with strong color deviation from their surroundings (purple or green VAE color bleed, for instance) are accepted as noise regardless of the cliff test result.

A blob is kept if either test fires. Per-blob component statistics $\left(\bar{L}^*, L^*_{peak}, \overline{a^*}, \overline{b^*}\right)$ are computed in bulk via np.bincount over the label image in a single pass. Only the per-blob background ring statistics require a Python loop, but since few blobs survive to this stage, the loop is typically short.

4.5 Stage 5 — Morphological Dilation

The verified mask is dilated by a user-specified radius (default 2 pixels) using an isotropic elliptical structuring element. This step is necessary because Telea inpainting samples pixel values from outside the masked region as source data. If the mask boundary sits right on the edge of the corrupted zone, the inpainter pulls in partially-damaged pixels and re-deposits them as "clean" paint, producing visible halos. Padding the mask by a few pixels ensures the inpaint boundary is firmly inside the clean neighborhood.

5. Inpainting Strategy

After detection, the masked regions are repaired using the Telea Fast Marching Method (FMM). Telea models the fill problem as a boundary-value PDE: starting from the mask boundary, pixel values propagate inward along isophote (equal-brightness) curves, producing smooth continuations of surrounding gradients. The inpaint radius is derived from $\max\left(2, \min\left(7, \lceil\sqrt{A}\rceil\right)\right)$.

The choice of Telea over PatchMatch-based or neural inpainting is deliberate. For the 1–5 pixel holes typical of VAE noise, Telea offers three advantages: (1) it runs in milliseconds because computation is proportional to masked area rather than whole-image search; (2) it produces deterministic results, which matters for reproducible workflows; and (3) it fills locally from immediate neighbors, avoiding the PatchMatch failure mode of importing unrelated textures from distant image regions. For larger damaged areas, neural inpainting would be superior, but the target use case is explicitly tiny isolated artifacts.

6. Interactive Tools

6.1 Standalone GUI Preview

A tkinter-based GUI application allows users to load images, individually or as folder batches, and interactively adjust all three parameters via sliders with real-time visual feedback. The application provides:

- 11 switchable views covering every intermediate stage of the pipeline, enabling transparent inspection of what each processing step contributes.
- A brush-based mask editor (left-click to add, right-click to erase) with adjustable radius, allowing manual correction of false positives and missed spots before saving the repaired result.
- A zoom lens and whole-canvas zoom/pan for examining fine details at high magnification.
- A statistics HUD displaying blob count, pixel coverage, area distribution, and detection time.

Parameter changes are debounced (80 ms) to prevent redundant pipeline re-runs during slider drags. All rendering is performed on clean algorithm output, with no UI text or overlays are baked into saved images.

6.2 ComfyUI Browser-Based Mask Editor

The VAENoiseMaskEditor node integrates a full-featured mask editor directly into the ComfyUI browser interface. When the user clicks "Edit Mask," a full-screen canvas overlay opens showing the original image with the detected mask as a red overlay. Features include brush and eraser tools, adjustable brush size and overlay opacity, undo/redo, mask clear and invert operations, and zoom/pan navigation. Edited masks are transmitted to the backend via REST API and picked up by the next workflow execution.

7. Limitations and Future Work

- When VAE corruption is severe, the max_noise_size budget may need manual increase, as the structural filter is calibrated for isolated small defects.
- Very small stars (≤ 2×2 pixels) that lack any internal luminance gradient may be misclassified as noise, since the cliff steepness metric requires at least some spatial extent to distinguish hills from cliffs.
- At 4K+ resolution, the CCA and per-blob LAB loop add 1–2 seconds of CPU time. Future work could offload the energy extraction stage to GPU via CUDA-accelerated filters while keeping the geometric analysis on CPU.
- The current system operates purely in pixel space and cannot address the root cause in latent space. Exploring latent-space regularization techniques could complement the post-processing approach.

8. Conclusion

This project demonstrates that a carefully engineered traditional computer vision pipeline can solve a targeted image defect problem more efficiently and transparently than neural alternatives. By decomposing the detection task into complementary energy extraction, structural filtering, and perceptual verification stages, the system achieves robust noise–highlight discrimination while maintaining millisecond performance and zero GPU dependency. The modular architecture, comprehensive preview system, and multiple interaction modes (ComfyUI node, standalone GUI, browser editor) make it practical for real production workflows where SDXL-generated images need quick, reliable artifact cleanup without sacrificing creative control.

References

[1] Rombach, R., et al. "High-Resolution Image Synthesis with Latent Diffusion Models." CVPR 2022.

[2] Podell, D., et al. "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis." arXiv:2307.01952, 2023.

[3] Telea, A. "An Image Inpainting Technique Based on the Fast Marching Method." Journal of Graphics Tools, 9(1):23–34, 2004.

[4] OpenCV Documentation. "Inpainting — cv2.inpaint." docs.opencv.org.

[5] CIE. "Colorimetry — Part 4: CIE 1976 L*a*b* Colour Space." CIE 015:2004.

Demo video: https://www.youtube.com/watch?v=qqUrk4OyEfI
Source code: https://github.com/megrez33281/ComfyUI-VAE-Noise-Fix